

HisabDo AI Copilot

Chatbot / AI Assistant Requirements Specification

Project: HisabDo Website Enhancement

Department: AI/ML

Module: Chatbot / AI Assistant Requirements

Prepared By: Muhammad Taha — AI/ML Team Lead

Document Type: Technical Requirements & Initial Implementation Specification

1. Purpose

This document defines the functional, technical, security, and behavioral requirements for the **HisabDo AI Copilot**.

The purpose of the AI Assistant is not to act as a generic chatbot. It should work as an intelligent assistant integrated with the HisabDo ecosystem and support users with:

- Product information
- FAQs
- Feature guidance
- Help and support
- Financial query routing
- Multilingual interaction
- Context-aware conversations
- Safe and verified responses

The assistant must follow the core architecture principle:

AI understands the user and explains information.

Verified application systems provide financial facts and calculations.

2. Chatbot Objectives

The HisabDo AI Assistant should provide a simple and intelligent interface through which users can interact with the HisabDo platform.

Primary Objectives

2.1 Provide Product Assistance

The assistant should help users understand the HisabDo platform and its available functionality.

Example queries:

- What is HisabDo?
 - What can I do with HisabDo?
 - How does the Khata feature work?
 - How can I add a customer?
-

2.2 Provide Feature Guidance

The AI Assistant should guide users through available features and workflows.

Supported areas may include:

- Customer management
- Khata and ledger management
- Transactions
- Expenses
- Receivables
- Reports
- PDF export
- Backup and restore
- Analytics
- Voice-related features

The assistant should provide guidance based on verified product information.

2.3 Answer FAQ and Help Questions

The chatbot should answer common support questions by retrieving information from the HisabDo knowledge base.

Potential knowledge sources include:

FAQs
+
Website Content
+
Feature Documentation
+

Help Articles
+
Product Guides
+
User Guides
+
Support Information

The assistant should not rely only on general AI knowledge when answering product-specific questions.

2.4 Support Financial Queries Safely

The AI Assistant may understand financial questions such as:

- How much does Ali owe me?
- What are my expenses this month?
- Who has the highest outstanding balance?
- What is my total receivable amount?

However, the AI model must **not independently generate or calculate critical financial values**.

Instead, the workflow should be:

```
User Financial Query
      ↓
Intent Detection
      ↓
Entity Extraction
      ↓
Verified Financial API
      ↓
Application Database / Business Logic
      ↓
Verified Financial Result
      ↓
AI Explanation
      ↓
User Response
```

The AI is responsible for understanding and explaining.

The application remains the source of truth.

3. Supported User Query Categories

The initial AI Assistant should classify requests into clear categories.

Proposed Intent Categories

PRODUCT_INFORMATION

FAQ_QUERY

FEATURE_GUIDANCE

REPORT_HELP

BACKUP_HELP

CUSTOMER_BALANCE_QUERY

TRANSACTION_QUERY

EXPENSE_QUERY

RECEIVABLE_QUERY

PAYABLE_QUERY

BUSINESS_INSIGHT

RECOMMENDATION_QUERY

GENERAL_CONVERSATION

UNKNOWN

Each intent should be routed to the appropriate module.

4. Supported Query Examples

Product Information

What is HisabDo?

What features does HisabDo provide?

Can HisabDo work offline?

Feature Guidance

How do I add a customer?
How can I record an expense?
How do I generate a report?

FAQ Queries

How does backup work?
Can I restore my data?
How do I export a PDF?

Financial Queries

How much does Ali owe me?
What were my expenses this month?
Which customer has the highest pending balance?

General Conversation

The assistant may support limited conversational interaction.

Example:

Hello
Can you help me?

The assistant should remain focused on the HisabDo ecosystem and should not unnecessarily behave as a general-purpose AI.

5. FAQ and Product Support Requirements

The chatbot should use a controlled knowledge retrieval process.

Knowledge Flow

User Question
↓
Input Processing
↓
Intent Classification
↓
Knowledge Search
↓

Relevant FAQ / Documentation
↓
Verified Context
↓
AI Response Generation
↓
User Response

Potential knowledge categories:

/knowledge-base

```
|— faqs
|— features
|— guides
|— documentation
|— help
|— website-content
```

Before information is added to the knowledge base, it should be reviewed for:

- Accuracy
- Relevance
- Clarity
- Currency
- Product correctness

Unverified information should not be treated as an authoritative source.

6. Financial Query Requirements

Financial queries require additional safety controls.

Required Rules

The AI Assistant must:

- Detect financial-related intents
- Extract relevant entities such as customer names
- Identify the required financial operation
- Request data through controlled backend APIs

- Use verified application data
- Respect authentication and authorization
- Ensure data belongs to the requesting user
- Prevent direct database access by the LLM
- Clearly handle unavailable financial data

Example Flow

User:

Ali ka kitna udhaar hai?

Processing:

Intent:
CUSTOMER_BALANCE_QUERY

Entity:
Ali

Required Tool:
Customer Balance API

Backend response:

```
{  
  "success": true,  
  "customer": "Ali",  
  "balance": 25000,  
  "currency": "PKR"  
}
```

AI response:

Ali ka current outstanding balance PKR 25,000 hai.

The amount must come from verified application logic, not from AI-generated assumptions.

7. Multilingual Support Requirements

The initial chatbot should support:

- English
- Roman Urdu
- Urdu as a future supported language

The priority for the initial POC should be:

English
+
Roman Urdu

Language Behavior

The preferred rule is:

```
Input Language
  ↓
Language Detection / Normalization
  ↓
Intent Processing
  ↓
Response Generation
  ↓
Same or Appropriate User Language
```

Example:

Input:

How do I add a customer?

Response: English

Input:

Customer kesay add karun?

Response: Roman Urdu

8. Roman Urdu Normalization

Roman Urdu contains spelling variations.

For example:

udhar
udhaar
udharr

kitna
kitny
kitnay
paisa
paisay
pese

The input-processing layer should normalize common variations before intent detection.

Example Normalization Flow

```
Raw Input
  ↓
Text Cleaning
  ↓
Lowercasing
  ↓
Whitespace Normalization
  ↓
Common Roman Urdu Normalization
  ↓
Processed Query
  ↓
Intent Detection
```

The system should preserve the user's natural language while improving internal understanding.

9. Conversation and Context Requirements

The AI Assistant should support controlled short-term conversation context.

Example:

User:

Ali ka balance batao.

Assistant:

Ali ka current outstanding balance PKR 25,000 hai.

User:

Uski last transaction kab hui thi?

The system should understand that:

"Uski" → Ali

Context Information

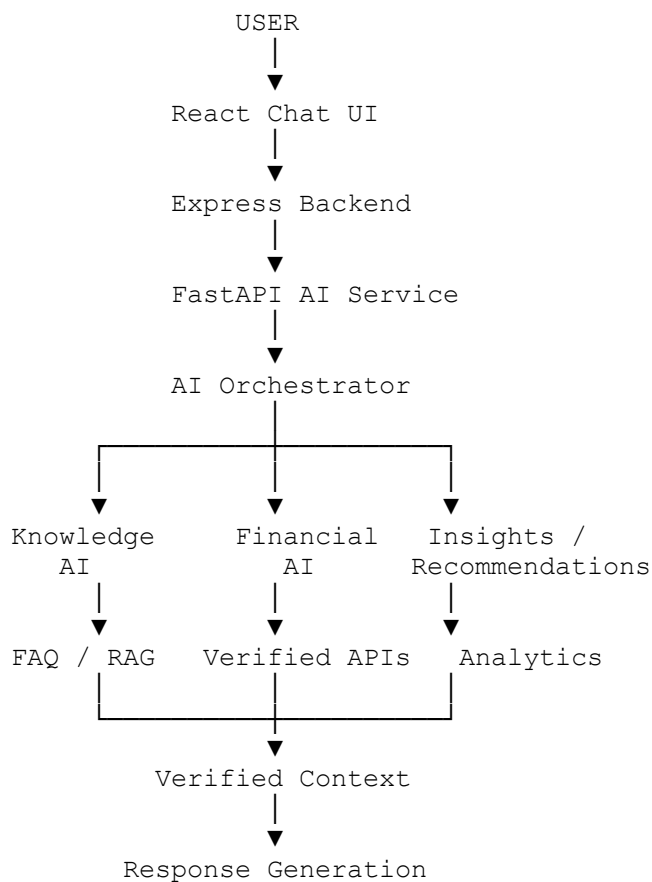
The system may temporarily maintain:

- Conversation ID
- Recent messages
- User language
- Current intent
- Selected customer context
- Session metadata

Sensitive financial information should not remain stored unnecessarily.

10. AI Assistant Architecture

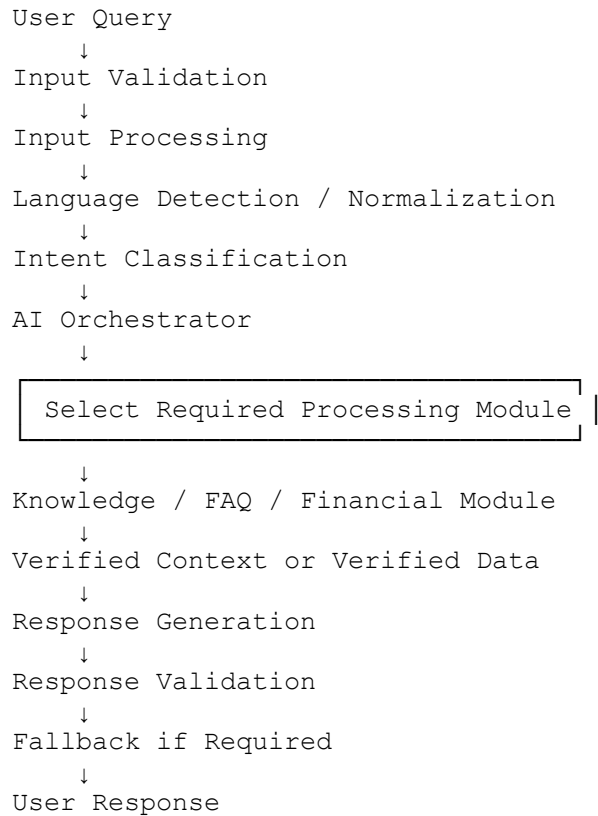
The proposed architecture is:



↓
User Response

11. Complete Request Workflow

Every user request should follow a controlled workflow.



12. AI Orchestrator Requirements

The AI Orchestrator acts as the central routing layer.

Its responsibility is to determine:

What is the user asking?

Which intent matches the request?

Which module should handle the request?

Is verified data required?

Is knowledge retrieval required?

Can the AI safely answer?

Example Routing

User Query Type	Destination
What is HisabDo?	Knowledge / FAQ Module
How do I add a customer?	Feature Guidance / Knowledge Base
How does backup work?	FAQ / RAG Module
How much does Ali owe me?	Financial API
What are my expenses?	Financial Data Service
What should I check today?	Recommendation / Insight Module
Unsupported question	Fallback Handler

13. Security and Privacy Requirements

Because HisabDo handles financial information, the chatbot must follow strict security rules.

Authentication

All authenticated AI requests involving user-specific information should include verified user identity.

The AI service should not trust user-provided identifiers without backend validation.

Authorization

Before accessing financial information, the backend should verify:

```
Authenticated User
  ↓
Permission Check
  ↓
User-Scoped Data Access
  ↓
Verified Result
```

A user must never receive another user's financial data.

No Direct LLM Database Access

The AI model must never receive unrestricted database access.

Incorrect approach:

```
LLM
↓
MongoDB
```

Recommended approach:

```
LLM / AI Orchestrator
↓
Controlled API / Tool
↓
Authentication
↓
Authorization
↓
Business Logic
↓
Database
```

This ensures that database access remains controlled and auditable.

14. Anti-Hallucination Requirements

The assistant must not invent information.

Product Questions

Use:

```
Verified Knowledge
+
Retrieved Documentation
+
Controlled Response Generation
```

Financial Questions

Use:

Verified Financial API
+
Application Business Logic
+
Structured Result
+
AI Explanation

The assistant should avoid unsupported statements.

If reliable information is unavailable, it should say so clearly.

Example:

I could not find verified information for this request.
Please rephrase your question or check the relevant HisabDo section.

15. Fallback Behavior

Fallback behavior is required for situations where the system cannot safely process a request.

Unknown Intent

Example:

I'm not sure I understood that request. Please rephrase it or ask me about HisabDo features, accounts, transactions, expenses, reports, or support.

Knowledge Not Available

I could not find verified information related to your question.

The system should avoid generating a confident but unsupported answer.

Financial Data Unavailable

I couldn't retrieve the requested financial information at the moment. Please try again later.

The AI must never estimate or invent missing balances.

AI Service Failure

The backend should return a user-friendly response without exposing internal errors.

Incorrect:

```
FastAPI connection refused on port 8000.
```

Correct:

```
The AI assistant is temporarily unavailable. Please try again shortly.
```

16. Proposed API Requirements

Chat API

POST /api/ai/chat

Request

```
{
  "message": "Ali ka kitna udhaar hai?",
  "conversationId": "conversation_id",
  "language": "roman-urdu"
}
```

Response

```
{
  "success": true,
  "type": "financial_answer",
  "language": "roman-urdu",
  "message": "Ali ka current outstanding balance PKR 25,000 hai.",
  "source": {
    "type": "verified_financial_data"
  }
}
```

AI Health API

GET /api/ai/health

Purpose:

- Check AI service availability

- Support monitoring
- Verify communication between backend and AI service

Knowledge Search API

POST /api/ai/knowledge/search

Purpose:

- Search FAQs
- Search product documentation
- Retrieve relevant verified knowledge

17. Suggested Code Structure

For implementation, the AI module can follow this structure:

```
hisabdo-ai/
├── frontend/
│   └── chatbot-ui/
├── backend/
│   ├── routes/
│   │   └── aiRoutes.js
│   ├── controllers/
│   │   └── aiController.js
│   └── services/
│       └── aiService.js
├── ai-service/
│   └── app/
│       ├── main.py
│       ├── api/
│       │   └── chat.py
│       ├── orchestrator/
│       │   └── router.py
│       ├── intents/
│       │   └── classifier.py
│       └── multilingual/
```



```
├── normalization.py
├── knowledge/
│   └── retrieval.py
├── financial/
│   └── tools.py
├── prompts/
│   └── system_prompt.py
├── utils/
│   └── validation.py
└── docs/
    └── chatbot-requirements.md
```

18. Basic Implementation Example

The following example demonstrates the **architecture pattern** for an initial POC.

FastAPI Chat Endpoint

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    message: str
    conversationId: str | None = None
    language: str | None = None

@app.post("/ai/chat")
async def chat(request: ChatRequest):

    message = request.message.strip()

    if not message:
        return {
            "success": False,
            "type": "invalid_input",
            "message": "Please enter a valid question."
        }

    # Step 1: Input processing
    normalized_message = message.lower()

    # Step 2: Intent classification
    intent = classify_intent(normalized_message)
```

```
# Step 3: Route through AI orchestrator
response = route_request(
    intent=intent,
    message=normalized_message
)

return {
    "success": True,
    "intent": intent,
    "response": response
}
```

Basic Intent Classification Example

```
def classify_intent(message: str) -> str:

    if any(word in message for word in [
        "what is hisabdo",
        "what does hisabdo do"
    ]):
        return "PRODUCT_INFORMATION"

    if any(word in message for word in [
        "how do i",
        "how to",
        "kesay",
        "kaise"
    ]):
        return "FEATURE_GUIDANCE"

    if any(word in message for word in [
        "balance",
        "udhar",
        "udhaar",
        "paisa"
    ]):
        return "FINANCIAL_QUERY"

    return "UNKNOWN"
```

For production, rule-based intent detection can later be replaced or enhanced with a more robust classifier or LLM-based intent layer.

Basic AI Orchestrator Example

```
def route_request(intent: str, message: str):

    if intent == "PRODUCT_INFORMATION":
        return handle_product_query(message)
```

```
if intent == "FEATURE_GUIDANCE":
    return handle_feature_guidance(message)

if intent == "FINANCIAL_QUERY":
    return handle_financial_query(message)

return {
    "type": "fallback",
    "message": (
        "I'm not sure I understood your request. "
        "Please ask me about HisabDo features, "
        "transactions, expenses, reports, or support."
    )
}
```

19. Functional Requirements Summary

The initial AI Assistant should provide:

- AI Chat Interface
- +
- Product Information
- +
- FAQ Support
- +
- Feature Guidance
- +
- English Support
- +
- Roman Urdu Support
- +
- Input Normalization
- +
- Intent Classification
- +
- AI Orchestration
- +
- Controlled Knowledge Retrieval
- +
- Safe Financial Query Routing
- +
- Conversation Context
- +
- Fallback Handling
- +
- Authentication Support
- +
- User-Scoped Financial Data
- +
- Error Handling

20. Non-Functional Requirements

The system should also focus on:

Performance

- Fast response for common queries
- Controlled timeout behavior
- Lightweight POC architecture

Reliability

- Graceful fallback during failures
- No exposure of internal system errors
- Retry behavior where appropriate

Security

- Authentication before sensitive data access
- Authorization for user-specific information
- No direct LLM database access
- User-scoped financial data

Scalability

The architecture should allow future addition of:

Advanced RAG
+
Additional Languages
+
Voice AI
+
Recommendation Engine
+
Predictive Insights
+
Local AI
+
Offline AI Optimization

21. Testing Requirements

The chatbot should be tested against the following categories:

Product Information
FAQ Queries
Feature Guidance
Financial Queries
English Queries
Roman Urdu Queries
Unclear Queries
Invalid Input
Unknown Queries
Knowledge Retrieval Failure
Financial API Failure
AI Service Failure
Fallback Responses

Each test should record:

- User query
- Expected intent
- Actual intent
- Expected behavior
- Actual response
- Pass/Fail status
- Identified issue

22. Initial Development Scope

For the initial POC, the recommended scope is:

Included

Basic Chat UI
+
Chat API
+
Input Validation
+
Basic Intent Detection
+
English Support
+
Roman Urdu Support
+
FAQ / Product Knowledge
+
Basic AI Orchestrator
+
Fallback Handling
+
Sample Financial Query Routing

Future Scope

Full RAG Pipeline
+
Vector Database
+
Advanced Multilingual Support
+
Complete Financial Intelligence
+
Personalized Recommendations
+
Dashboard Insights
+
Voice AI
+
Predictive Analytics
+
Offline AI Optimization

23. Final Requirement Principle

The HisabDo AI Assistant should operate according to the following principle:

```
graph TD
    A[User Question] --> B[Understand the Request]
    B --> C[Identify Intent]
    C --> D[Route to Correct Module]
    D --> E[Retrieve Verified Knowledge or Data]
    E --> F[Generate Controlled Response]
    F --> G[Validate Response]
    G --> H[Provide Safe User Answer]
```

The chatbot should **not become the source of truth.**

Instead:

AI understands the user.

Knowledge systems provide verified product information.

Financial APIs provide verified financial data.

Application logic performs calculations.

The AI explains the result clearly to the user.

Final Expected Outcome

After implementing these requirements, HisabDo will have a clear foundation for an AI Assistant capable of providing:

- Intelligent product support
- FAQ assistance
- Feature guidance
- English and Roman Urdu interaction
- Context-aware conversations
- Safe financial query handling
- Controlled AI responses
- Secure integration with the HisabDo ecosystem
- A scalable foundation for future RAG, recommendations, insights, voice AI, and offline AI capabilities